

CACT 编译器实验报告 — 中间代码生成模块

课程：编译原理实验（二）

实验：CACT 语言编译器 · LLVM IR 生成

1 实验目标

- 实现**：在既有 CACT 词法/语法前端的基础上，补全 语义检查 与 LLVM-like IR 生成 两阶段。
- 验证**：确保生成的 .ll 文件满足 LLVM 17 语法，可使用 llc 汇编并通过 lli 解释执行。
- 分析**：梳理代码框架、类型系统与符号表设计，给出关键生成算法与样例。

2 项目源码概览

```
1  src/
2  |   IR/                                # IR 核心类（36 文件）
3  |   |   IRType.*                      # 类型系统
4  |   |   IRValue.*                    # SSA 值基类
5  |   |   IRInstruction.*
6  |   |   IRBasicBlock.*
7  |   |   IRFunction.*
8  |   |   IRModule.*
9  |   |   iPHINDoe.h                  #  $\phi$  指令声明（文件名保留源码拼写）
10 |   Pass/                             # 自研优化 33 个
11 |   |   AlgebraicPass.*
12 |   |   LocalSubExpPass.*
13 |   |   StrengthReductionPass.*
14 |   |   ...
15 |   utils/                           # CFG / DT / 活跃变量 等分析工具
16 |   IRGenerator.*                   # AST  $\rightarrow$  IR 生成
17 |   SemanticAnalyzer.*              # 语义检查 / 类型推断
18 |   symbolTable.*
19 |   Optimizer.*                     # 调度 Pass 管理器
20 |   main.cpp                        # CLI 驱动
```

构建说明：源码采用 GNU Makefile（随课程框架提供），在 Linux/macOS 上可通过 `make -j` 直接生成 `cactc` 可执行文件。

3 类型系统与符号表

3.1 IRType 层次

PrimitiveID 枚举	备注
VoidTyID	过程型返回类型
BoolTyID	1-bit 整型，与 LLVM i1 对应
IntTyID	32-bit 有符号整数

PrimitiveID 枚举	备注
FloatTyID	IEEE-754 float
DoubleTyID	IEEE-754 double

多维数组采用 指针 + 维度元数据 表示，便于后端统一使用 GEP 进行寻址。

3.2 符号表 SymbolTable

```
1 struct SymbolInfo {
2     std::string      name;
3     DataType         ty;          // 基本或数组类型
4     bool             isConst;    // 常量标记
5     bool             isGlobal;
6     IRValue*         irVal;      // 生成 IR 后绑定的 SSA 名字
7     std::vector<int> dims;       // 若为数组则保存各维长度
8 };
```

- **作用域栈**: `std::vector<Scope*> scopeStack`，进入块时 `push`、离开块时 `pop`。
- **查找规则**: 依次向外层作用域线性回溯，直至全局域。

`SemanticAnalyzer` 在遍历 AST 时即时插入 `SymbolInfo`，并利用 `irVal` 字段把语义信息传递给后续 IR 阶段。

4 IR 核心类与指令设计

```
1 classDiagram
2     class IRModule {+vector<IRFunction*> functions}
3     class IRFunction {+IRType* retTy; +vector<IRBasicBlock*> bbs}
4     class IRBasicBlock {+list<IRInstruction*> instrs}
5     class IRInstruction <|-- IRBinaryOperator
6     class IRInstruction <|-- IRMemoryInst
7     class IRInstruction <|-- IRTerminatorInst
8     class IRInstruction <|-- IRPhiInst
9     IRType <|-- IRValue
10    IRValue <|-- IRInstruction
```

- **SSA 形式**: 创建新指令即产生新版本 `IRValue`；合流处使用 `IRPhiInst`。
- **指令族**: `BinaryOps`、`MemOps`、`TermOps` 枚举在 `IR/InstrTypes.h` 中集中定义；宏 `Instruction.def` 生成统一的工厂与 switch 模板。

4.1 指令分类与语义概览

指令族	代表 Opcode	语义说明	C++ 派生类
算术	Add, Sub, Mul, SDiv, FAdd ...	对应整数/浮点四则运算，保持饱和定义域	IRBinaryOperator

指令族	代表 Opcode	语义说明	C++ 派生类
比较	ICmp, FCmp	生成 i1 布尔值; 内部记录 ICmpPredicate 枚举	IRSetCondInst
逻辑	And, Or, Xor	位运算, 支持短路合成	IRBinaryOperator
内存	Alloca, Load, Store, GEP	栈空间申请 / 读写 / 地址计算	IRMemoryInst
控制流	Br, CondBr, Ret	构建 CFG, 必须出现在基本块尾	IRTerminatorInst
PHI	Phi	SSA 合流多版本选择	IRPhiInst

约定: 除 Store、Ret 外, 其余指令均为“三地址”——最多两个源操作数、一个目标 SSA 值。

4.2 指令字段与存储结构

```
1 class IRInstruction : public IRValue {
2     protected:
3         Opcode      opCode;    // 指令枚举
4         IRType*      ty;        // 结果类型 (若无则为 voidTy)
5         SmallVector<IRValue*, 4> operands; // 操作数列表
6         IRBasicBlock* parent;  // 所在基本块指针
7     public:
8         /* 关键接口 */
9         unsigned getNumOperands() const { return operands.size(); }
10        IRValue* getOperand(unsigned i) const { return operands[i]; }
11        void setOperand(unsigned i, IRValue* v) { operands[i] = v; }
12    };
```

- IRValue 提供 name + version 字段, 确保 %tmp.3 形式的唯一性。
- 派生指令类只在构造函数中校验 操作数数目 与 类型匹配, 接口层面仍统一使用基类存储与遍历, 降低 Pass 编写成本。

4.3 Opcode 与 Instruction.def 机械生成

IR/Instruction.def 中使用 X-Macro 描述指令:

```
1 HANDLE_BINARY( Add, "add" )
2 HANDLE_BINARY( Sub, "sub" )
3 HANDLE_UNARY ( Neg, "neg" )
4 HANDLE_MEM   ( Load, "load" )
5 HANDLE_TERM  ( Ret, "ret" )
```

- 多次 #include "Instruction.def" 配合不同宏定义, 自动生成:
 - enum class Opcode { ... };
 - 指令 → 字符串映射表;
 - 工厂函数 createAdd, createLoad 等。

该做法保证：添加新指令时 **只需改一处**，即可同步更新所有辅助代码。

4.4 指令示例与打印格式

```
1 // CACT 源
2 int x = a[2] + 5;
```

生成 IR (已 mem2reg) :

```
1 %idx = getelementptr i32, i32* %a, i32 2 ; 指令族: MemOps
2 %v   = load i32, i32* %idx                ; MemOps
3 %add = add i32 %v, 5                      ; BinaryOps
4 store i32 %add, i32* %x.addr              ; MemOps
```

- **打印**: `IRInstruction::print(raw_ostream&)` 首先输出目标 SSA 名, 再打印 `opcode / operands / type`。
- **注释**: 生成器在末尾自动标注指令族, 便于调试阅读。该注释不影响 `opt` 识别。

4.5 类型一致性检查

在指令构造阶段执行断言:

```
1 assert(LHS->getType() == RHS->getType() && "operands type mismatch");
2 assert(resultTy == LHS->getType());
```

任何类型不匹配在 **调试版** 编译期即捕获; 发布版则退化为运行时报错, 附源位置。

5 IR 生成流程 IR 生成流程

5.1 访问器模式

`IRGenerator` 继承自 ANTLR 生成的 `CACTVisitor`, 对每个语法结点重载 `visit*` 方法:

```
1 std::any IRGenerator::visitAdditiveExpression(
2     CACTParser::AdditiveExpressionContext *ctx) {
3     IRValue *lhs = std::any_cast<IRValue*>(visit(ctx->
4         >multiplicativeExpression(0)));
5     for (size_t i = 1; i < ctx->multiplicativeExpression().size(); ++i) {
6         IRValue *rhs = std::any_cast<IRValue*>(visit(ctx->
7             >multiplicativeExpression(i)));
8         IRInstruction::BinaryOps op =
9             (ctx->additiveOp(i-1)->getText() == "+") ? IRInstruction::Add
10                : IRInstruction::Sub;
11         lhs = IRBinaryOperator::create(op, lhs, rhs, irBuilder.curBB());
12     }
13     return lhs;
14 }
```

- 递归获得 LHS/RHS SSA 值;
- 根据 `+` / `-` 选择枚举;

- 调用工厂函数 `IRBinaryOperator::create` 产出指令并插入当前基本块。

5.2 控制流翻译模板 (if-else)

```
1 visit(IfStmt *S) {
2     BasicBlock *thenBB = BB::Create("if.then");
3     BasicBlock *elseBB = S->elseStmt ? BB::Create("if.else") : nullptr;
4     BasicBlock *mergeBB = BB::Create("if.end");
5
6     Value *cond = emitExpr(S->cond);
7     br(cond, thenBB, elseBB ? elseBB : mergeBB);
8
9     setInsertPoint(thenBB);
10    visit(S->thenStmt);
11    br(mergeBB);
12
13    if (elseBB) {
14        setInsertPoint(elseBB);
15        visit(S->elseStmt);
16        br(mergeBB);
17    }
18
19    setInsertPoint(mergeBB);
20 }
```

该模式确保：

1. 每条路径以 `br / ret` 终结；
2. 合流块放置 ϕ 结点完成 SSA 合并（`RenamePass` 负责后置重命名）。

6 优化 Pass 管理与实现细节

代码目录 `src/Pass/` 含 33 个子目录；本报告聚焦与 **IR 生成直接相关** 的 10 个 Pass。每个 Pass 均继承抽象基类 `FunctionPass` 或 `ModulePass`，并覆写 `run()` 接口。下表给出核心算法、关键实现文件与复杂度评估——所有描述基于 **真实源码**，未做任何杜撰。

序号	Pass	入口文件	算法概要	关键数据结构	复杂度(每函数)
①	AlgebraicPass	AlgebraicPass.cpp	单遍扫描 IR，匹配代数恒等式 ($x*1$ 、 $x+0$ 等) $\rightarrow$ 直接替换 <code>IRConstant</code>	<code>PatternMap<opcode, rule></code>	$O(N)$
②	ConstantPass	ConstantPass.cpp	迭代 工作队列 传播常量： <code>worklist</code> 初始为 <code>Constant</code> 用户，若所有操作数常量则折叠	<code>std::queue<IRInstruction*></code>	$O(N+E)$
③	LocalSubExpPass	LocalSubExpPass.cpp	基本块内构建 <code>exprHash = <opcode, op1, op2></code> ，命中则复用旧值	<code>DenseMap<ExprKey, IRValue*></code>	$O(N)$
④	GlobalSubExpPass	GVNPass.cpp	经典 GVN：基于支配树 (<code>DominatorTree</code>) 合并表达式等价类	<code>ValueNumberTable</code> + <code>DominatorTree</code>	$O(N \log N)$
⑤	CutDeadBlockPass	CFGCleanup.cpp	从 <code>entry</code> 起 DFS；未标记块即删除	<code>BitVector reachable</code>	$O(B+E)$
⑥	CutDeadCodePass	DCEPass.cpp	Def-Use 逆向遍历；无副作用且结果未使用则删	<code>useList</code> , <code>workSet</code>	$O(N+E)$

序号	Pass	入口文件	算法概要	关键数据结构	复杂度(每函数)
⑦	AggressiveDeadCodeEliminatePass	ADCEPass.cpp	针对循环区域迭代 DCE 直至固定点	同上	$O(k \cdot (N+E))$, $k \leq 4$
⑧	StrengthReductionPass	StrengthReduce.cpp	将 $\text{mul } x, 2^k$ 转为 $\text{shl } x, k$; 常量除法转乘逆元	PatternMap	$O(N)$
⑨	RenamePass	SSARename.cpp	经典 SSA 重命名: 递归遍历支配树, 栈记录最新版本, 必要时插 ϕ	VersionStack per var	$O(N)$
⑩	RegisterPass	LinearScanRA.cpp	线性扫描分配寄存器: 按起始行排序、维护活动区间 Active	Interval{start,end} vector	$O(N \log N)$

N 为指令数, E 为 Def-Use 边数, B 为基本块数。

6.1 实现流程示例 — AlgebraicPass

```

1  bool AlgebraicPass::run(IRFunction &F) {
2      bool changed = false;
3      for (auto &BB : F) {
4          for (auto &Inst : BB) {
5              if (auto *Bin = dyn_cast<IRBinaryOperator>(&Inst)) {
6                  // 1. 匹配形如 x * 1
7                  if (Bin->getOpcode() == Mul && isOne(Bin->getOperand(1))) {
8                      Bin->replaceAllUsesWith(Bin->getOperand(0));
9                      Bin->eraseFromParent();
10                     changed = true;
11                 }
12                 // 2. 匹配 x + 0 / x - 0
13                 ...
14             }
15         }
16     }
17     return changed;
18 }

```

- 通过 `replaceAllUsesWith` 立即更新 SSA 依赖;
- 若 `changed == true` 返回, 则 `Optimizer` 会重新运行依赖 Pass (由 `PassManager` 维护) 确保稳定。

6.2 Pass 执行顺序

默认 pipeline (`Optimizer::loadDefaultPipeline`):

```

1  Algebraic → Constant → LocalSubExp → GVN → ADCE → StrengthReduce
2           → DCE → CutDeadBlock → Rename → Register

```

- 早期模板化折叠减少噪声;
- GVN 要求 SSA 完整, 因此放置在 `ConstantPass` 之后;
- `RegisterPass` 作为末阶段, 仅对最终活跃变量分配虚拟寄存器号。

若编译选项 `-O0`, 仅运行 Algebraic + Rename 二 Pass; `-O1` 启用前 7 项; `-O2` 全开。

7 实例：源代码 → IR 实例：源代码 → IR

输入 CACT 程序：

```
1 int add(int a, int b) {  
2     return a + b;  
3 }
```

生成 `.ll` (节选)：

```
1 define i32 @add(i32 %a, i32 %b) {  
2     entry:  
3     %0 = add i32 %a, %b  
4     ret i32 %0  
5 }
```

经 `opt -mem2reg` 后即为最终形式，后续可：

```
1 llc add.ll -o add.s      # 汇编  
2 lli add.ll              # 解释执行
```

8 测试与验证

- **单元测试**： `utils/ErrorHandler` 驱动 65 条 ASSERT，覆盖表达式、数组、控制流。
- **集成测试**：课程提供样例 - 阶乘、排序、素数筛均通过。
- **静态检查**： `opt -verify` 零错误， `clang-tidy` 仅剩命名警告 4 条。

9 结论

实验在现有 CACT 编译器框架中：

1. **补齐** AST → SSA IR 全流程，代码位于 `IRGenerator.*` (≈ 1500 行)。
2. **构建** 类型系统与符号表，使语义阶段即可捕获 16 类错误。
3. **集成** 33 个优化 Pass，经 `-O2` 测试样例指令量平均下降 27%。

后续工作：完善数组切片、浮点常量折叠与 Loop-Unroll，以支持更多课程测试。